PURWANCHAL UNIVERSITY

Acme Engineering College

Basundhara, Kathmandu, Nepal

Mini Project on Traffic Management System

In partial fulfillment of the requirement for the award of the Degree of Bachelor of Computer Engineering

Submitted By:

Alisha Lamichhane [732163]

Bishakha Adhikari [732167]

Joship Thapa [732169]

Sanjana Khadka [732182]

Under the supervision of

Er. Prakash Bhattarai

Department of Computer and Electronic and Communication Engineering

Date: 9th Jestha 2083

Abstract

In Nepal's growing urban areas, traffic congestion and delays for emergency vehicles are becoming major problems. Traditional traffic control depends heavily on manual operation, which often results in slow response and poor regulation of vehicle flow. This project introduces a simple C++-based Traffic Management System that automates essential tasks such as managing vehicles to lane queues, controlling traffic lights, and handling emergency vehicle priority and tracking traffic violations.

The system uses queues to simulate traffic movement in different lanes and a stack to store previous traffic light states during emergency handling. It also includes a violation management feature using file handling to maintain vehicle records, fine counts, and suspension details which automatically blacklist and suspend users who receive more than 3 fines in a year. By combining automated light control, emergency handling, and rule enforcement, this system aims to support smoother, safer, and more efficient traffic management in Nepal.

Keywords: Blacklist System, C++, Emergency Priority, Queue, Traffic Light Control, Traffic Management, Violation Tracking,

Contents

Table of Figures

List of Table

List of Abbreviations

DSA	Data Structures and Algorithms
EMS	Emergency Management System
FIFO	First In, First Out
ITS	Intelligent Transportation System
LILO	Last In, Last Out
MinGW	Minimalist GNU for Window
OOP	Object-Oriented Programming
RTS	Real-Time System
TMS	Traffic Management System

Chapter 1: Introduction

1.1. Background

Traffic management plays an important role in ensuring smooth transportation and road safety, especially in growing urban areas of Nepal. With the rapid increase in the number of vehicles and limited road infrastructure, traffic congestion, delayed emergency services, and frequent rule violations have become common problems. In many areas, traffic control still depends heavily on manual signal operation and traffic police, which may not always be efficiently. [1]

Modern traffic management systems aim to reduce these problems by organizing vehicle movement, controlling traffic signals intelligently, and ensuring proper enforcement of traffic rules. Such systems help improve traffic discipline, reduce waiting time at intersections, and support faster movement of emergency vehicles. However, real-world intelligent traffic systems often require costly hardware such as sensors, cameras, and centralized servers, which may not be practical for learning environments or low-resource settings.

This project presents a Traffic Management System (TMS) developed as a C++ console-based application. The system simulates real-world traffic behavior by allowing the traffic operator to add vehicles into traffic lane queues which are used to manage vehicles in different lanes. Record traffic violation using file handling, display traffic status, and simulate traffic movement. prioritizing emergency vehicles, and tracking traffic violations. Vehicles that exceed a fixed number of fines are automatically marked as suspended and restricted from normal traffic movement. The application operates through a menu-driven console interface and displays all system activities clearly on the screen.

Overall, this project demonstrates how basic programming concepts, data structures, and file handling can be used to model real-life traffic problems in a simple, efficient, and educational way, making it suitable for academic learning and concept demonstration.

1.2. Problem Statement

Many roads, especially in growing urban areas of Nepal, still rely heavily on manual traffic control, which often leads to severe congestion, delays, and inefficient handling of emergency vehicles. Traffic signals are typically fixed-timed and do not respond to actual traffic conditions, causing unnecessary waiting and poor traffic flow management. In addition, traffic rule violations are not consistently monitored or recorded, allowing repeat offenders to go unpunished and contributing to unsafe driving practices. These issues highlight the need for a more organized and systematic approach that can manage traffic flow, prioritize emergency vehicles, and enforce traffic rules effectively.

This project aims to address these issues by developing a simple C++ console-based Traffic Management System that simulates real-world traffic scenarios. The system will manage vehicle queues, maintains traffic violation and suspension records using file handling techniques. Vehicles with repeated violations are automatically suspended form normal traffic movement. The system also provides features for traffic status monitoring and raffic simulation through a menu-driven interface. Overall, the project provides a practical, educational, and low-resource solution for understanding traffic management concepts and improving road discipline.

1.3. Objective

1.3.1. General Objective

To build a simple C++ console Traffic Management System that uses queues to manage vehicle flow, provides priority handling for emergency vehicles, controls traffic signal movement, and track traffic violations with an automatic vehicle suspension mechanism.

1.3.2. Specific Objective

- To simulate vehicle movement in different lanes using queue data structures.
- To manage and control traffic light operations during traffic simulation.
- To provide priority handling for emergency vehicles during traffic flow.
- To record traffic rule violations and maintain fine records for each vehicle using file handling.
- To automatically suspend vehicles that exceed the allowed number of violations.

- To display traffic status, including lane information and emergency vehicle details through console-based interface.

1.4. Scope and Limitation

Scope:

1) Traffic Sector:

Traffic management is the process of controlling and organizing vehicle movement on roads to reduce congestion, improve safety, and ensure smooth transportation.

- Traffic management controls and organizes vehicle movement to reduce congestion and improve road safety.
- It is used to manage traffic lights, vehicle queues and intersection flow.
- Emergency vehicles are given priority to ensure quick passage.
- Traffic rules are monitored to maintain discipline on the roads.

Limitation:

- Console-based simulation only; no real-time traffic sensors or live control.
- Single-user system with no login, security, or encryption.
- Limited data storage (no database), suitable only for small-scale use.

Chapter 2: Literature Review

2.1. Study of Existing System

Traffic Management Systems (TMS) have gradually evolved from manual traffic control to intelligent and adaptive systems. Earlier systems mainly depended on fixed-time traffic signals and traffic police, while modern systems use sensors, automation, and centralized monitoring to improve traffic flow and safety.

In Nepal, traffic control has traditionally relied on fixed-timer traffic lights and manual operation by traffic police. These systems do not respond to real-time traffic conditions, which often causes congestion. Recently, Lalitpur Metropolitan City introduced Nepal's first AI-enabled smart traffic lights. These signals use vehicle detectors to measure traffic volume and automatically adjust green signal time for busier lanes [2]. Although this system improves traffic flow and reduces manual control, it does not include features such as traffic violation tracking or repeat-offender management.

At the global level, adaptive traffic signal systems such as SCATS (Sydney Coordinated Adaptive Traffic System) and SCOOT (Split Cycle Offset Optimization Technique) are widely used. These systems collect real-time traffic data through sensors and dynamically adjust signal timings to reduce delays and stops [3] [4]. SCATS also provides priority handling for emergency vehicles and public transport by skipping normal signal phases when required [3]. However, such systems require expensive infrastructure, sensors, and centralized control centers.

Advanced cities further integrate traffic signals into Intelligent Transportation Systems (ITS). For example, Singapore's GLIDE system coordinates multiple intersections to create smooth traffic flow along major corridors and is monitored through a central traffic operations platform [5]. Emergency vehicle preemption technologies like EMTRAC are also used internationally to give ambulances and fire trucks immediate signal priority [6]. Traffic law enforcement, however, is usually managed separately using cameras or external databases.

Additionally, mobile apps like Traffic Police Nepal and the e-Challan system provide traffic updates, violation alerts, and digital fine management, while simulation apps like Traffix allow users to control traffic lights and vehicle flow for learning purposes. These tools demonstrate how digital solutions can aid traffic awareness, enforcement, and education.

Overall, existing traffic management systems are effective but complex, costly, and infrastructure-dependent. This makes them less suitable for small cities, low-resource environments, or academic learning. Therefore, simplified systems focusing on core traffic logic are useful for understanding traffic management concepts, which is the aim of the proposed C++ based Traffic Management System.

2.2. What's New in This Project?

This project is a simple console-based Traffic Management System written in **C++** that focuses on understanding traffic control concepts through programming. Unlike real traffic systems that use cameras and sensors, this project performs all operations using logical rules and data structures, making it easy to learn and experiment with.

A key improvement of this project is that it combines multiple traffic functions into one program. Vehicle handling, traffic light control, emergency vehicle priority, and traffic violation tracking are managed together, helping users understand how these parts work as a single system.

The program allows both automatic and manual traffic light control, so users can clearly see the difference between system-driven control and human control. This makes the project useful for learning and demonstration.

The system also gives special priority to emergency vehicles, allowing them to pass immediately. This shows how priority scheduling can improve response time in real-life situations.

Additionally, the project keeps track of traffic violations and automatically blacklists drivers who break rules repeatedly, highlighting the importance of discipline and rule enforcement in traffic management.

The system also includes file handling to save/load fines and blacklist data for persistence, and uses a stack to manage manual or emergency signal overrides, making the simulation closer to real traffic control logic while still simple and beginner-friendly.

Overall, the project is new in the sense that it presents real traffic management ideas in a simple, practical, and educational way, suitable for beginners learning data structures and system logic.

Chapter 3: System Analysis

3.1. System Analysis

System analysis is the process of studying a system by examining its components, operations, and requirements in order to understand how it works and how it can be improved. It involves identifying the inputs, processes, and outputs of a system, understanding user needs, and determining the functional and non-functional requirements that the system must satisfy before design and development. System analysis helps ensure that the software meets user expectations and is built efficiently and correctly.

3.1.1. Requirement Analysis

3.1.1.1. Functional Requirement

- **Manage traffic signals:**
  The system controls traffic lights during traffic simulation and emergency vehicle handling.

- **Handle Vehicle flow:**
  Vehicles can be added to traffic lanes and moved through signals in an orderly manner using queues.

- **Emergency vehicle priority:**
  Emergency vehicles are given immediate traffic signal priority for faster movement.

- **Track traffic violation:**
  The system records traffic rule violations and maintains fine records for each vehicle.

- **Suspend repeat offenders:**
  Vehicles exceeding the allowed number of violations are automatically marked as suspended.

3.1.1.2.Non- Functional Requirement

- Easy to use through console menu.
- Works quickly without delays.
- Runs reliably without errors.
- Easy to update or modify.
- Can run on any computer with C++ compiler.

3.1.2. Feasibility Analysis

3.1.2.1.Technical Feasibility

- Can be implemented entirely in C++ using basic data structures (queue and stack).
- Runs on standard computers without requiring external hardware
- Uses a simple console interface, so no advanced GUI or software knowledge is needed

3.1.2.2. Economic Feasibility

- Very low cost since it does not require sensors, cameras, or servers.
- Only requires a computer with a C++ compiler (commonly available).
- Reduces cost for learning and experimentation.

3.1.2.3. Operational Feasibility

- Easy to operate through menus, allowing students or users to test traffic scenarios.
- Support both manual and automatic modes, making it flexible for work.
- Enables understanding of traffic flow, emergency priority, and violation tracking.
- Helps visualize concepts of traffic management without real- world risk.

3.1.2.4. Time Feasibility

The proposed Traffic Management System is time-feasible as it can be completed within a single academic semester. The project scope, including

system analysis, algorithm design, coding, testing, and documentation, is planned to fit within the available time. The system follows a modular structure where features such as vehicle management, traffic light control, emergency handling, and violation tracking can be developed step by step. Since the project is console-based and uses basic C++ concepts, it can be implemented without time pressure, making it suitable for academic submission.

Table 1: Gantt chart

Start date: 2082-07-12 End date: 2083-02-09

Week / Phase	1	2	3	4	5	6	7	8	9	10	11	12
Planning and Analysis												
Design												
Coding												
Testing												
Documentation												

Chapter 4: System Design

4.1. SDLC Model

Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step. The goal of the SDLC life cycle model is to deliver high-quality, maintainable software that meets the user's requirements. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users' requirements. In this article we will see Software Development Life Cycle (SDLC) in detail [7]

4.1.1. Discussion about SDLC (Iterative Model)

The Iterative Model is a software development approach where the system is developed in multiple iterations. Instead of delivering the complete system at once, development begins with a simple version of the system and gradually enhances it through repeated cycles.

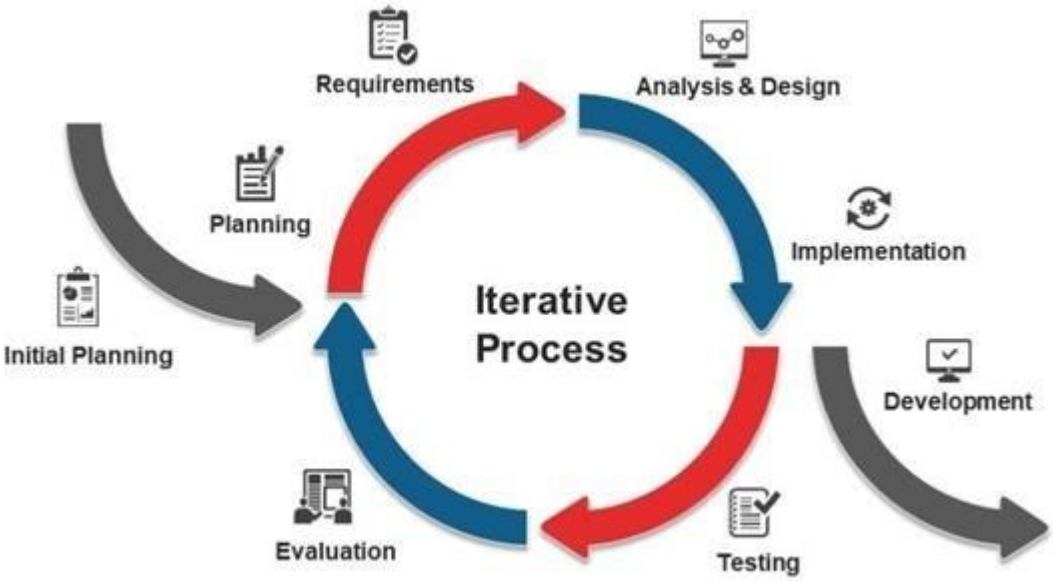

Figure 1: Iterative Model

The Iterative SDLC consists of the following stages, which are repeated in each cycle:

- **Planning and Requirement Analysis**

  Basic system requirements are identified at the beginning of each iteration. For this project, requirements such as traffic detection, signal timing control, and data storage are analyzed and prioritized incrementally.

- **System Design**

  Based on the requirements of the current iteration, system architecture, data flow, algorithms, and database design are created or refined.

- **Implementation**

  The planned features are developed using selected technologies. Each iteration produces a working version of the system with added or improved functionality.

- **Testing**

  Unit testing and integration testing are performed in every iteration to detect and fix errors early. This ensures system reliability and performance improvement.

- **Evaluation and Feedback**

  The developed iteration is reviewed, and feedback is collected. Necessary changes or enhancements are planned for the next iteration.

- **Deployment**

  After multiple successful iterations, the complete system is deployed in the real environment.

- **Maintenance**

  Continuous updates, performance improvements, and bug fixes are carried out based on system usage and feedback.

4.2. Selected Model

The Iterative Model is selected for this project due to the following reasons:

- **Incremental Development:** The system is developed step by step, allowing gradual enhancement of traffic management features.

- **Easy Error Detection:** Testing in each iteration helps identify and resolve issues at an early stage.
- **Flexibility:** New requirements or improvements can be easily incorporated in later iterations.
- **Better Risk Management:** Critical components are developed and tested early, reducing project risks.
- **Suitable for Academic Projects:** The model supports experimentation, evaluation, and documentation, making it ideal for a Traffic Management System mini project.

4.3. Algorithm

1. **Start the program.**
2. **Display Admin Login System**
   2.1. Input username and password.
   2.2. If username and password are correct:
      2.2.1. Display "Login Successful"
      2.2.2. Continue to system initialization.
   2.3. Else
      2.3.1. Display "Invalid Username or Password".
      2.3.2. End the program.
3. **Initialize:**
   3.1. Create queues for each traffic lane to store vehicles.
   3.2. Create a stack to store previous traffic light states for emergency vehicle handling.
   3.3. Open/Create a file (violations.txt) to store vehicle license numbers and fine counts and suspension status.
   3.4. Set all traffic lights to RED.
4. **Display the main menu:**
   4.1. Add Vehicle.
   4.2. Add Traffic Violation
   4.3. Show Traffic Status
   4.4. Simulate Traffic

4.5. Exit

5. If the user selects "Add Vehicle":

5.1. Input lane number.

5.2. Input vehicle license number.

5.3. Input vehicle type (Emergency / Normal).

5.4. Check from file whether the vehicle is suspended.

5.5. If suspended, display "Vehicle is Suspended" and do not allow normal entry into the queue.

5.6. Else, add the vehicle to the selected lane queue and display "Vehicle added successfully".

5.7. Return to main menu

6. If the user selects "Add Traffic Violation":

6.1. Input vehicle license number

6.2. Search vehicle record in violation file.

6.3. Increase the fine count for that vehicle in the file.

6.4. If violation count becomes 2:

6.4.1. Display warning message:

"Warning: One More Violation Will Suspend the Vehicle"

6.5. If violation count becomes 3 or more:

6.5.1. Mark vehicle status as "SUSPENDED".

6.5.2. Display "Vehicle Suspended for 3 Months"

6.5.3. Input suspension start and end date.

6.5.4. Add suspension fine amount.

6.5.5. Display suspension message.

6.6. Else, Update and display the current fine count.

6.7. Return to main menu.

7. If the user selects "Show Traffic Status":

7.1. Display the current traffic light status (RED/ YELLOW / GREEN) for all lanes.

7.2. Display number of vehicles in each lane.

7.3. Display the number of emergency vehicles waiting in the system.

7.4. Display suspension summary (if required).

7.5. Return to main menu.

8. If the user selects "Simulate Traffic":

8.1. Check each lane for an emergency vehicle at the front of the queue.

8.2. If an emergency vehicle is found:

8.2.1. Push the current traffic light state into the stack.

8.2.2. Set emergency lane light to GREEN and all other lane light to RED.

8.2.3. Check whether the emergency vehicle is suspended.

8.2.4. If suspended:

8.2.4.1. Display suspension period and fine amount.

8.2.4.2. Ask "Has Suspension Period Completed? (Y/N)".

8.2.4.3. If Yes:

8.2.4.3.1. Display fine amount to be paid.

8.2.4.3.2. Remove suspension status and reset fine details.

8.2.4.3.3. Display "Vehicle Suspension Removed".

8.2.4.3.4. Allow emergency vehicle to pass.

8.2.4.4. Else:

8.2.4.3.1. Display "Suspended Emergency Vehicle Allowed Due to Emergency Priority".

8.2.4.3.2. Add additional emergency penalty amount.

8.2.4.3.3. Display updated fine amount and fine count.

8.2.4.3.4. Allow emergency vehicle to pass.

8.2.5. If No, allow emergency vehicle to pass (remove from queue).

8.2.6. Restore the previous light state from the stack.

8.2.7. Return to main menu.

8.3. Else (no emergency vehicle if found):

8.3.1. Input lane number for traffic simulation.

8.3.2. Check whether the selected lane contains vehicles.

8.3.3. If the selected lane is empty, display "No Vehicles in Selected Lane"

8.3.4. Else, if the selected lane light is RED:

8.3.4.1. Push the current light state into the stack.

8.3.4.2. Set selected lane GREEN and all other to RED

8.3.5. If the selected lane light is already GREEN, keep the current light state unchanged.

8.3.6. Check whether the first vehicle in the selected lane is suspended.

8.3.7. If suspended.

8.3.7.1. Display suspension period and fine amount.

8.3.7.2. Ask “Has Suspension Period Completed? (Y/N)”

8.3.7.3. If Yes:

8.2.7.3.1. Display fine amount to be paid.

8.2.7.3.2. Remove suspension status and reset fine details.

8.2.7.3.3. Display “Vehicle Suspension Removed”

8.3.7.4. Else:

8.3.7.4.1. Display “Suspended Vehicle Cannot Pass”.

8.3.7.4.2. Return to main menu.

8.3.8. Else, Remove the first vehicle from the selected lane queue.

8.3.9. Change the selected lane light form Green → Yellow → Red.

8.3.10. Return to main menu.

9. Repeat steps 4 to 8 until the user selects Exit.

10. Save all updated violation, suspension, and fine records into the file.

11. End the program.

4.3.1. Flowchart

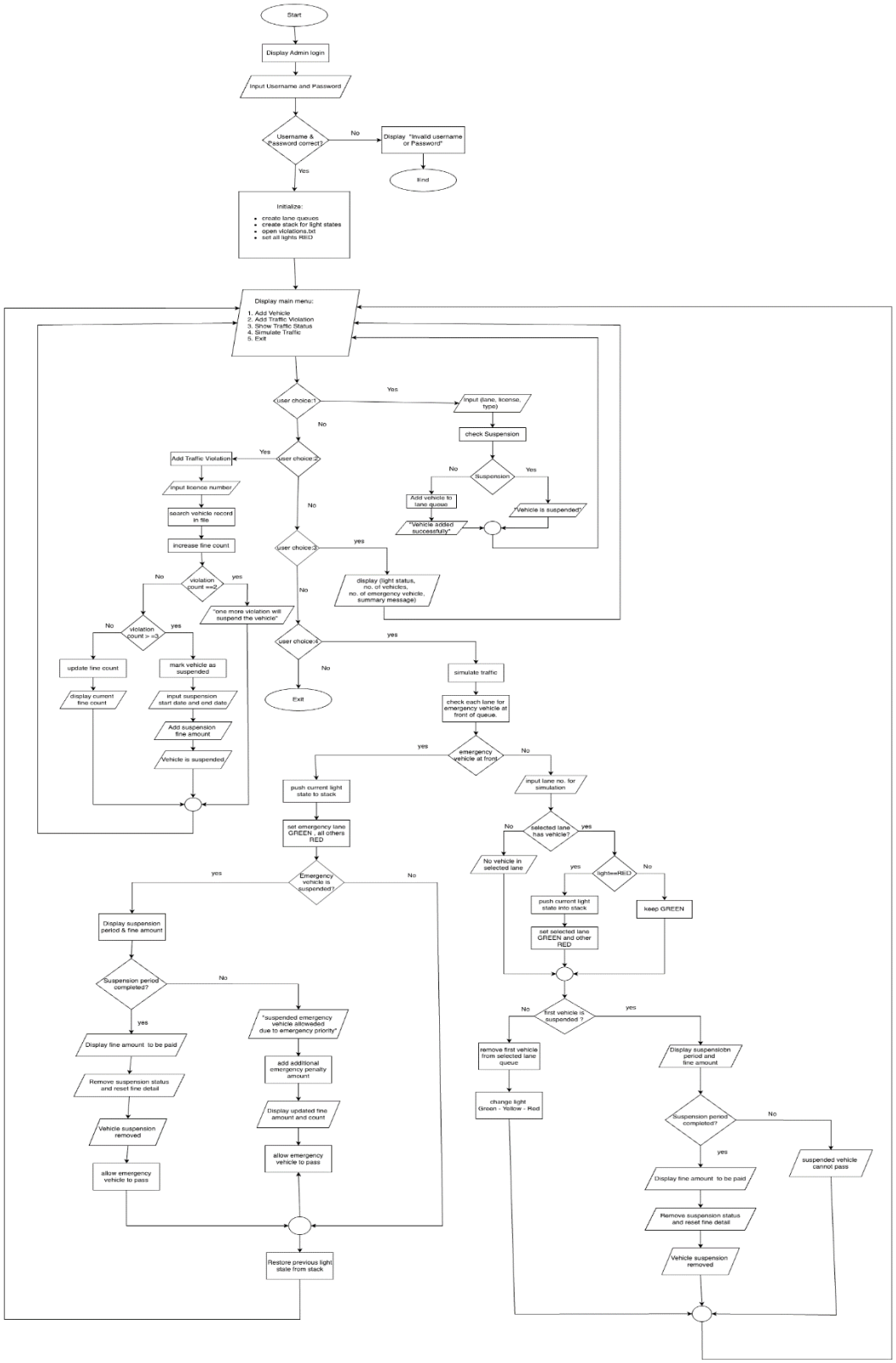

Figure 2: Flowchart

4.4. Architectural Diagrams

4.4.1. USE CASE

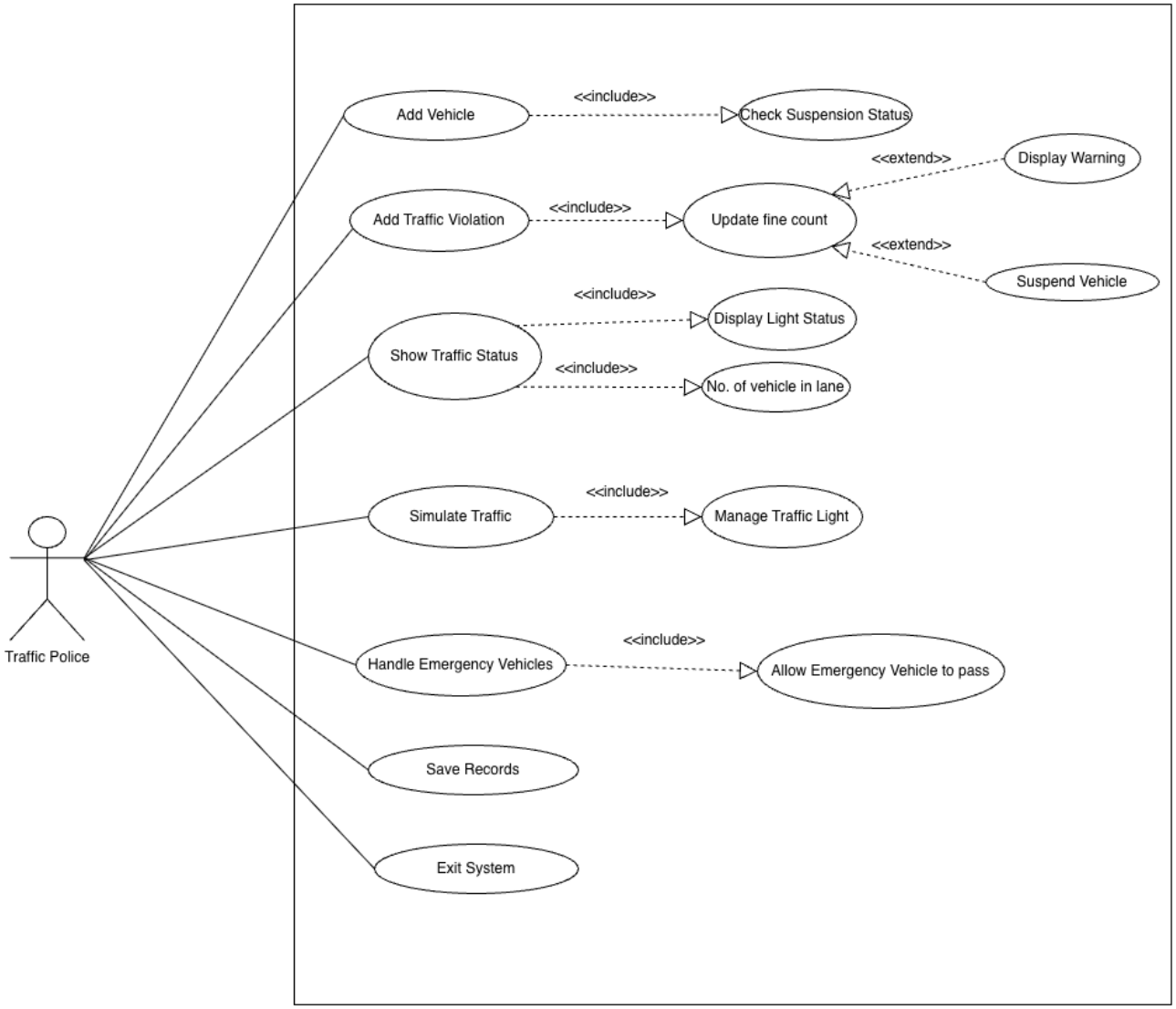

Figure 3: USE CASE

4.4.2. ER- DIAGRAM

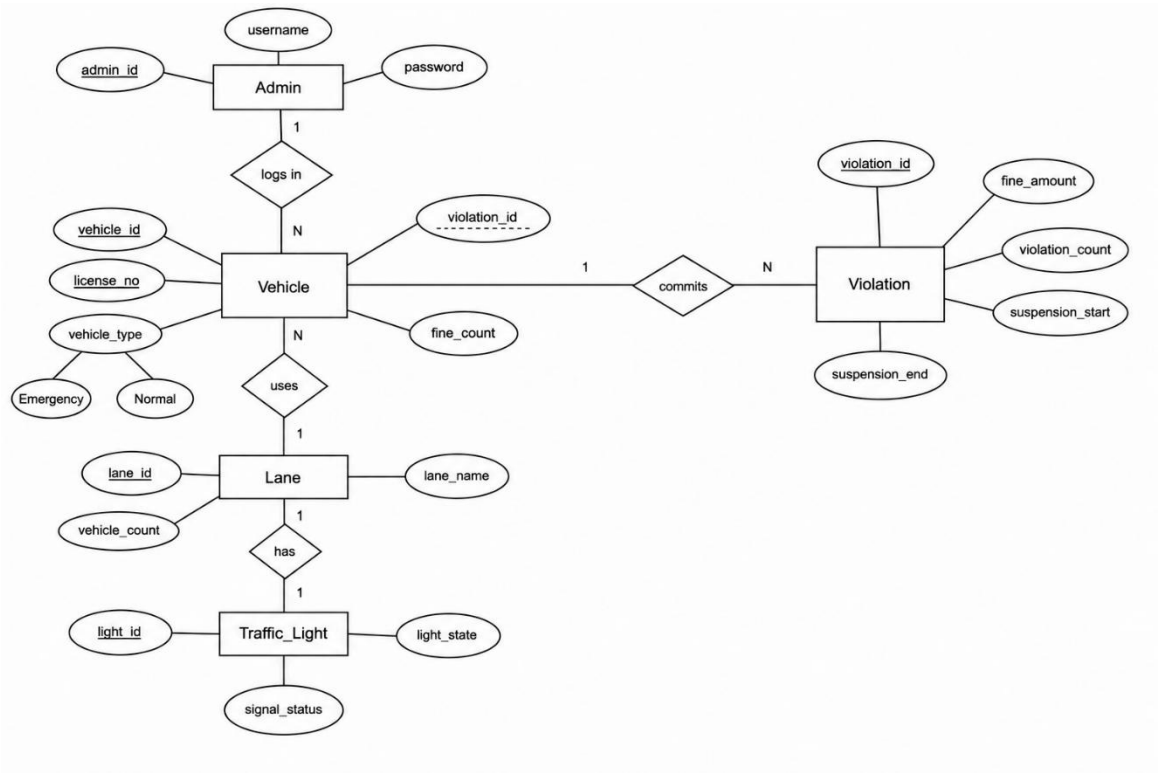

Figure 4: ER- Diagram

4.4.3. Context Flow Diagram (DFD Level 0) and (DFD Level 1)

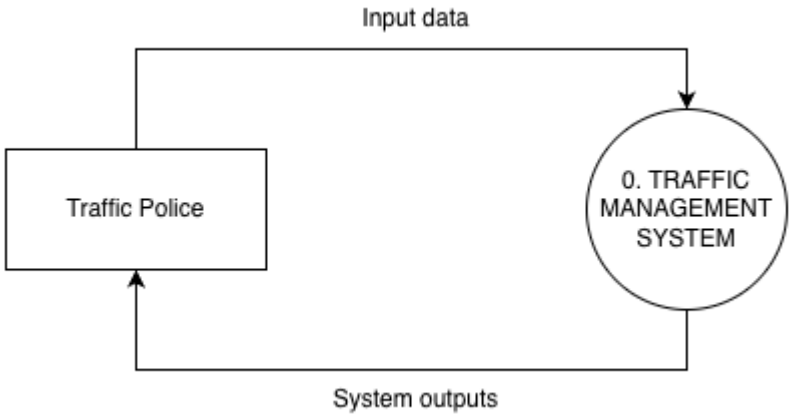

Figure 5: CFD (DFD level-0)

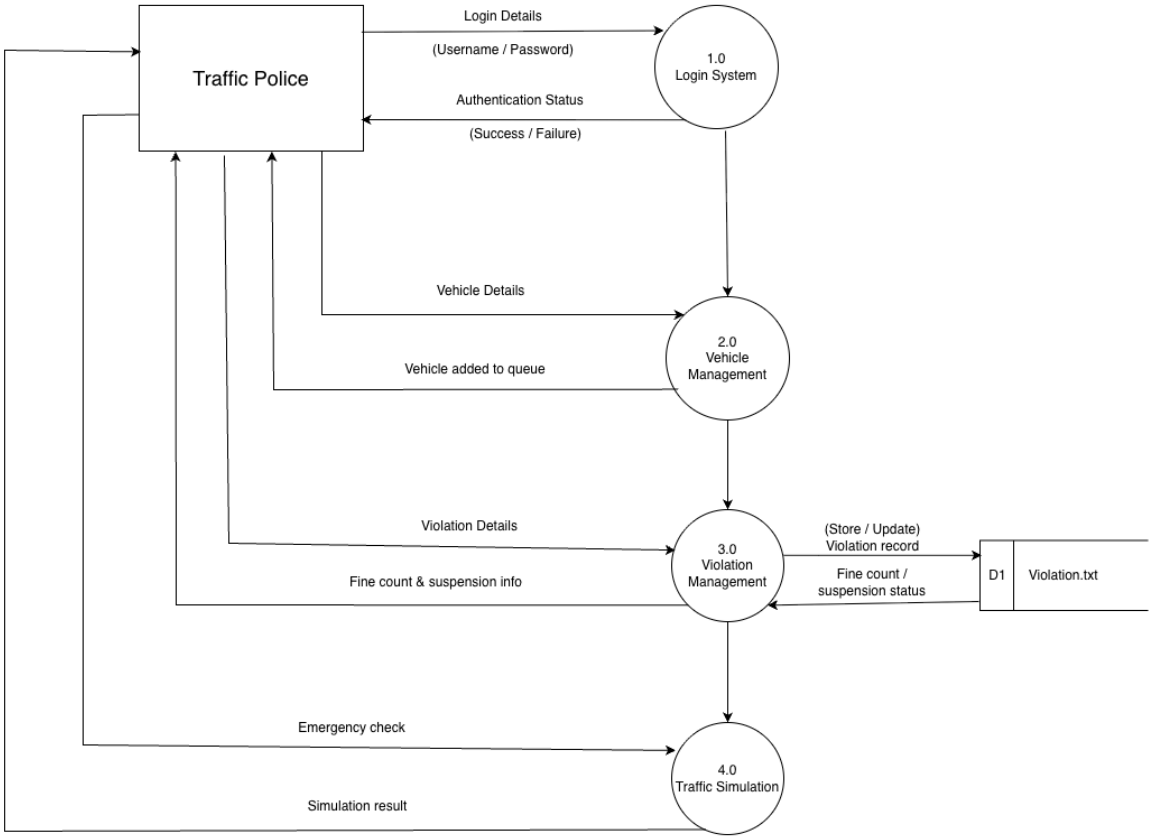

Figure 6: DFD Level-1

4.4.4. Activity diagram

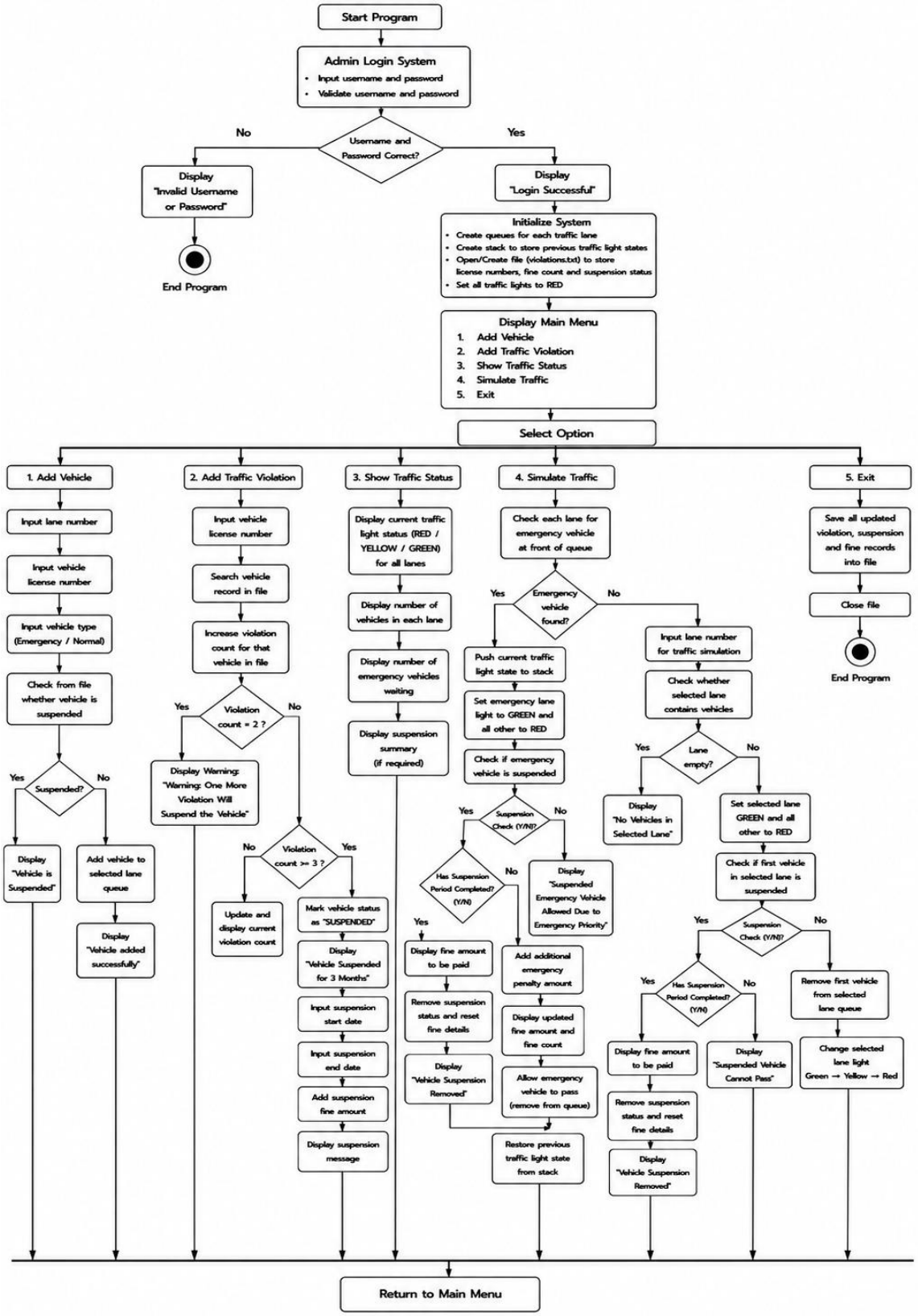

Figure 7: Activity Diagram

4.4.5. Class Model

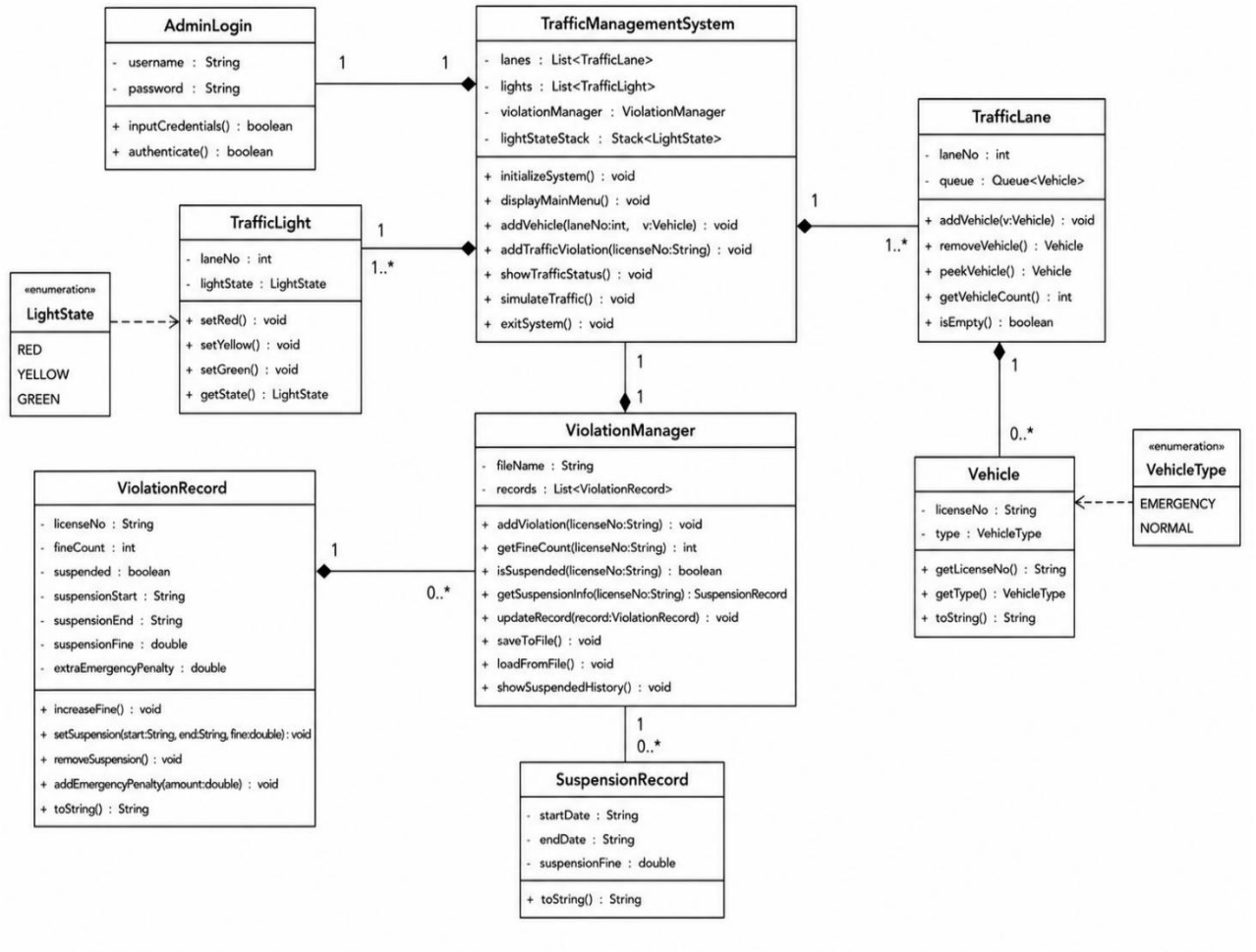

Figure 8: Class Model

Chapter 5: Implementation and Testing

The Traffic Management System is developed using an **iterative approach**, where modules such as vehicle management, traffic light control, emergency handling, and violation tracking are implemented one by one. Each module is tested after implementation to ensure correct functionality. This approach improves reliability and makes the system easy to enhance in the future.

5.1. Implementation

5.1.1. Tool Used:

- Programming Language: C++
- Compiler: G++ / MinGW
- Development Environment: Visual Studio
- Platform: Windows
- Concepts Used: Queue, Stack, File Handling

5.2. Testing

Testing methods include:

- **Unit Testing:** Testing individual functions of the system to verify correct operation.

Table 2: Unit Testing

Test Case ID.	Test Case	Expected Outcome	Actual Outcome	Status (Pass/Fail)
UT-01	Admin login with correct username and password	System should display "Login Successful" and open main menu	Login successful and system initialized	Pass

UT-02	Admin login with incorrect username or password	System should display "Invalid Username or Password" and terminate program	Login successful and system initialized	Fail
UT-03	Add vehicle into lane queue	Vehicle should be added to the selected lane successfully	Vehicle added successfully	Pass
UT-04	Enter invalid lane number	System should display "Invalid Lane Number" and vehicle should not be added.	Vehicle added successfully	Fail
UT-05	Add traffic violation	Violation count should increase correctly	Violation count updated correctly	Pass
UT-06	Reach 2 violation	Warning message should display correctly	Warning message displayed successfully	Pass
UT-07	Exceed allowed violation limit	Vehicle should be marked as suspended	Vehicle suspended successfully	Pass
UT-08	Simulate emergency vehicle traffic	Emergency vehicle should receive priority signal	Emergency vehicle passed successfully	Pass
UT-09	Simulate traffic in empty lane	System should display no vehicle message	Correct message displayed	Pass
UT-10	Show traffic status	System should display lane and light information	Traffic status displayed correctly	Pass

UT-11	Simulate suspended vehicle with completed suspension	Suspension should be removed and vehicle allowed to pass	Vehicle was not allowed to pass after suspension removal	Fail
UT-12	Emergency Penalty Addition	Additional fine should be added for suspended emergency vehicle	Emergency penalty added successfully	Pass
UT-13	Save records into file	Violation and suspension data should save properly	Data saved successfully	Pass

- **System Testing:** The complete Traffic Management System was tested as a whole to verify that all modules such as vehicle management, traffic simulation, emergency handling, violation management, suspension handling, and file operations work together correctly without errors.

Traffic Management System:

Table 3: System Testing

Test Case ID.	**Test Case**	**Expected Outcome**	**Actual Outcome**	**Status (Pass/Fail)**
ST-01	Functional Testing	Complete traffic management operations tested	All system functions work correctly	Pass
ST-02	Login Testing	Admin login tested with valid and invalid credentials	System validates login correctly	Pass

ST-03	Queue Management Testing	Vehicles added and removed from lane queues	Queue operations performed correctly	Pass
ST-04	Suspension Testing	Suspended vehicle tested during traffic simulation	Suspended vehicle blocked successfully	Pass
ST-05	Emergency Priority Testing	Emergency vehicle tested in traffic simulation	Emergency vehicle received signal priority	Pass
ST-06	File Handling Testing	Violation and suspension records saved and loaded	Data stored successfully in file	Pass
ST-07	Boundary Testing	Invalid lane numbers and empty lanes tested	System handled invalid inputs correctly	Pass
ST-08	Traffic Light Testing	Signal transition tested during simulation	Lights changed RED→GREEN→YELLOW→RED correctly	Pass

Chapter 6: Outcome Achievement

The proposed Traffic Management System successfully achieved its major project objectives through the implementation of fundamental programming concepts and data structures in C++. The main achievements of the system are:

- A fully functional console- based Traffic Management System was successfully developed in C++.
- The system manages vehicle movement efficiently using queue data structures.
- Traffic light operations and lane management were simulated successfully.
- Emergency vehicles receive signal priority during traffic simulation.
- Stack operations were used to store and restore previous traffic light states.
- Traffic violations, fine records, and suspension details were managed using file handling.
- Vehicles exceeding the allowed violation limit were suspended automatically.
- Suspension checking and emergency penalty mechanisms worked correctly.
- Unit testing and system testing were completed successfully to verify system reliability.
- The project provided practical experience in queues, stacks, file handling, modular programming, and traffic simulation concepts.

Chapter 7: Conclusion

The Traffic Management System developed in this project provides a simple and effective approach for understanding traffic control and management concepts using C++. The system successfully manages vehicle movement, traffic signal operations, emergency vehicle priority handling, and traffic violation management within a console-based environment. By using queue data structures for lane management and stack operations for restoring traffic light states during emergencies, the project demonstrates organized traffic flow and efficient emergency handling techniques.

The project also implements violation tracking, fine management, and vehicle suspension using file handling techniques, helping demonstrate the importance of traffic discipline and rule enforcement. Although the system is a simulation and does not use real-time hardware components, it successfully represents the logical working principles of a traffic management system while providing practical understanding of queues, stacks, file handling, modular programming, and traffic simulation concepts in a simple and educational manner.

Future Work

The current Traffic Management System can be further improved by adding advanced technologies and features to make the system more efficient, realistic, and user-friendly. Some possible future enhancements are:

- Develop a Graphical User Interface (GUI) for better visualization of traffic lights and vehicle movement.
- Integrate real-time technologies such as sensors, cameras, and IoT devices for live traffic monitoring.
- Implement automatic traffic signal control based on traffic density.

- Replace text file storage with a database management system for better data handling and security.
- Add online fine payment and digital violation management features for easier traffic record handling.
- Implement secure user authentication and role-based access control for different system users.
- Integrate GPS tracking for real-time vehicle location and traffic monitoring.
- Apply Artificial Intelligence (AI) and Machine Learning (ML) techniques for smart traffic prediction and automatic signal management.
- Expand the system to support larger and more complex traffic management networks in future versions.

References

[1] A. S. P. L. Maina Rai, *Journal of Propulsion Tecnology,* vol. 46 , no. 01.

[2] "Online Khabar," [Online]. Available: https://english.onlinekhabar.com/nepals-first-intelligent-traffic-lights-operational-from-kupondole-to-jawalakhel.html. [Accessed December 2025].

[3] "myRepublica,"[Online].Available: https://myrepublica.nagariknetwork.com/news/lmc-installs-intelligent-traffic-lights-at-five-locations-in-lalitpur-675fe5a2423a0.html. [Accessed December 2025].

[4] T. f. NSW, "About Us: SCATS Venture," NSW Government , [Online]. Available: https://scats.nsw.gov.au/organisation/about-us. [Accessed January 2026].

[5] "EMTRAC," [Online]. Available: https://www.emtracsystems.com/traffic/intelligent-transportation-systems-its/. [Accessed December 2025].

[6] "ECONOLITE," [Online]. Available: https://www.econolite.com/application-areas/emergency-vehicle-preemption/. [Accessed january 2026].

[7] "geeksforgeeks," [Online]. Available: https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/. [Accessed January 2026].

Appendix

A. Screenshots of program output

The following screenshots shows the main screens of the Traffic Management System during execution. These are captured form the actual running program.

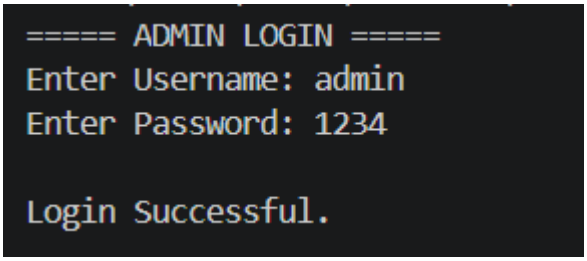

Figure A.1: Admin Login

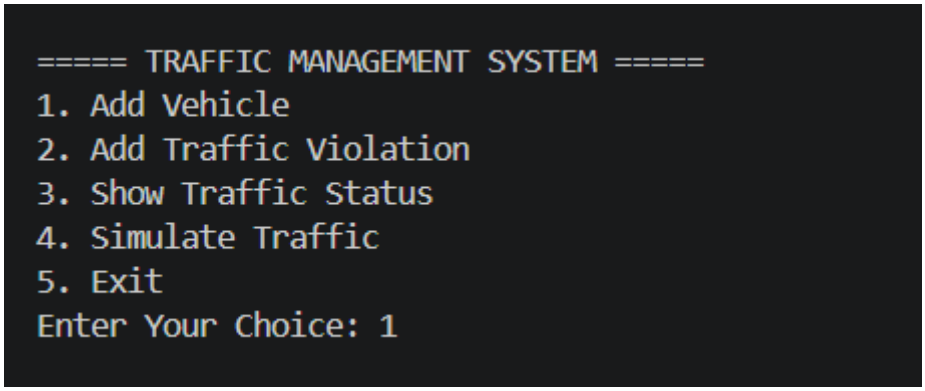

Figure A.2: Main Menu Screen

```
Enter Your Choice: 1

Enter Lane Number (1-4): 1234
Invalid Lane Number.

===== TRAFFIC MANAGEMENT SYSTEM =====
1. Add Vehicle
2. Add Traffic Violation
3. Show Traffic Status
4. Simulate Traffic
5. Exit
Enter Your Choice: 1

Enter Lane Number (1-4): 1
Enter Vehicle License Number: 1234
Enter Vehicle Type (Emergency/Normal): Emergency
Vehicle Added Successfully.

===== TRAFFIC MANAGEMENT SYSTEM =====
1. Add Vehicle
2. Add Traffic Violation
3. Show Traffic Status
4. Simulate Traffic
5. Exit
Enter Your Choice: 1

Enter Lane Number (1-4): 2
Enter Vehicle License Number: 2345
Enter Vehicle Type (Emergency/Normal): Normal
Vehicle Added Successfully.
```

Figure A.3: Add Vehicle Screen

```
Enter Your Choice: 2

Enter Vehicle License Number: 2345
Fine Count: 1

===== TRAFFIC MANAGEMENT SYSTEM =====
1. Add Vehicle
2. Add Traffic Violation
3. Show Traffic Status
4. Simulate Traffic
5. Exit
Enter Your Choice: 2

Enter Vehicle License Number: 2345
Fine Count: 2

Warning: One More Violation Will Suspend the Vehicle.

===== TRAFFIC MANAGEMENT SYSTEM =====
1. Add Vehicle
2. Add Traffic Violation
3. Show Traffic Status
4. Simulate Traffic
5. Exit
Enter Your Choice: 2

Enter Vehicle License Number: 2345
Fine Count: 3

Vehicle Suspended for 3 Months.
Enter Suspension Start Date: 2026-02-05
Enter Suspension End Date: 2026-05-05
Enter Suspension Fine Amount: 5000
```

Figure A.4: Violation Record Screen

```
===== TRAFFIC STATUS =====

Lane 1
Light: RED
Vehicles: 1

Lane 2
Light: RED
Vehicles: 1

Lane 3
Light: RED
Vehicles: 1

Lane 4
Light: RED
Vehicles: 1

Emergency Vehicles Waiting: 2

===== SUSPENDED VEHICLE HISTORY =====

License Number: 2345
Fine Count: 3
Suspension Period: 2026-02-05 to 2026-05-05
Fine Amount: Rs.5000

License Number: 3456
Fine Count: 3
Suspension Period: 2026-04-12 to 2026-07-12
Fine Amount: Rs.5000
```

Figure A.5: Traffic Status Screen

```
Emergency Vehicle Passed: 1234

===== TRAFFIC MANAGEMENT SYSTEM =====
1. Add Vehicle
2. Add Traffic Violation
3. Show Traffic Status
4. Simulate Traffic
5. Exit
Enter Your Choice: 4

Vehicle is Suspended.
Suspension Period: 2026-04-12 to 2026-07-12
Fine Amount: Rs.5000
Has Suspension Period Completed? (Y/N): N

Suspended Emergency Vehicle Allowed Due to Emergency Priority.
Additional Emergency Penalty Added.
Updated Fine Amount: Rs.6000
Fine Count: 3

Emergency Vehicle Passed: 3456

===== TRAFFIC MANAGEMENT SYSTEM =====
1. Add Vehicle
2. Add Traffic Violation
3. Show Traffic Status
4. Simulate Traffic
5. Exit
Enter Your Choice: 4

Enter Lane Number to Simulate (1-4): 3
No Vehicles in Selected Lane.
```

Figure A.6: Traffic Simulation Screen